

tted.

Logs, Events, and Span Events

Log messages can be sent using three different objects according to the OTEL specification:

- Log
- Event
- Span Event

At least in the first iteration we can only either support Logs, Events, or Span-Events.

We can't send Span Events as there are lots of legacy code that can not or will not implement tracing for various reasons.

Even though Events use the same data model internally, their semantics differ.

Logs have a mandatory severity level as a first-class parameter that Events do not need to have, and Events have a mandatory event.name and optional event.domain keys in the Attributes field of the Log record.

Further, logs typically have messages in string form and events have data in the form of key-value pairs.

There is a discussion to separate Log and Event APIs.

More information on the differences between these two can be found [here](#).

From the perspective of a developer/potential user, there seems to be no logging use case that couldn't be modeled as a Log record instead of sending an Event explicitly.

Examples that the community gives e.g. [here](#) or [here](#) are not convincing enough and could simply be modeled as Log records.

Hence the decision to only support Log objects seems like a boring and simple solution.

Rate-limiting

Similar to traces, logging data ingestion will be done at the Ingress level.

As part of the forward-auth flow, Traefik will forward the request to Gatekeeper which in turn leverages Redis for counting.

This is currently done only for the ingestion path.

Check the MR description for more details on how it works.

The read path rate limiting implementation is tracked [here](#).

Database schema

OpenTelemetry specification defines a set of fields that are required by the implementations.

There are some small discrepancies between the documented schema and the protobuf definition, namely, TraceFlags is defined as an 8-bit field in the documentation whereas it is a 32-bit wide field in the proto definition.

The remaining 24 bits are reserved.

The Log message body may be any object and there is no size limitation for the record.

For the purpose of this design document, we will assume that it is going to be an arbitrary string, either plain text or e.g. JSON, without length limits.

Data filtering

The schema uses Bloom Filters extensively.

They prevent false negatives, but false positives are still possible, hence we will not be able to provide != queries to users.

The Body field is a special case, as it uses tokenbfv1 tokenized Bloom Filter.

The tokenbfv1 skipping index seems like a simpler and more lightweight approach than the ngrambfv1 index.

Based on the very preliminary benchmarks below ngrambfv1 index will be also much more difficult to tune.

The limitation is though that our users will be able to search only the full words for now.

We (gu)estimate that there may be up to 10,000 different words in a given granule, and we aim for a 0.1% probability of false positives

Using this tool the optimal size of the filter was calculated at 143776 bits and 10 hash functions.

Skipping indexes, ==, != and LIKE operators

Skipping indexes only optimize searching for granules to scan.

== and LIKE queries work as they should, the != always results in a full scan due to Bloom Filters limitations.

At least in the first iteration we will not make != operator available to users.

Based on the data, it may be much easier for us to tune the tokenbfv1 filter in the first iteration than the ngrambfv1, because in the case of ngrambfv1 queries almost always result in a full scan for any reasonably big dataset.

The reason for that is that the number of ngrams in the index is much higher than tokens hence matches are more frequent for data with high cardinality of words/symbols.

A very preliminary benchmark was conducted to verify these assumptions.

As testing data, we used the following table schemas and inserts/functions.

They simulate a single tenant, as we want to focus only on the Body field.

Normally the primary index would allow us to skip granules where there is no data for a given tenant.

tokenbfv1 version of the table:

plaintext

```
CREATE TABLE tbl2
```

```
(
```

```
  Timestamp DateTime64(9) CODEC(Delta(8), ZSTD(1)),
```

```
  TraceId String CODEC(ZSTD(1)),
```

```
  ServiceName LowCardinality(String) CODEC(ZSTD(1)),
```

```
  Duration UInt8 CODEC(ZSTD(1)),
```

```
  SpanName LowCardinality(String) CODEC(ZSTD(1)),
```

```
  Body String CODEC(ZSTD(1)),
```

```
  INDEX idxbody Body TYPE tokenbfv1(143776, 10, 0) GRANULARITY 1
```

```
)
```

```
ENGINE = MergeTree
```

```
PARTITION BY toDate(Timestamp)
ORDER BY (ServiceName, SpanName, toUnixTimestamp(Timestamp), Traceld)
SETTINGS indexgranularity = 8192
```

ngrambfv1 version of the table:

```
plaintext
CREATE TABLE tbl3
(
  Timestamp DateTime64(9) CODEC(Delta(8), ZSTD(1)),
  Traceld String CODEC(ZSTD(1)),
  ServiceName LowCardinality(String) CODEC(ZSTD(1)),
  Duration UInt8 CODEC(ZSTD(1)),
  SpanName LowCardinality(String) CODEC(ZSTD(1)),
  Body String CODEC(ZSTD(1)),
  INDEX idxbody Body TYPE ngrambfv1(4,143776, 10, 0) GRANULARITY 1
)
ENGINE = MergeTree
PARTITION BY toDate(Timestamp)
ORDER BY (ServiceName, SpanName, toUnixTimestamp(Timestamp), Traceld)
SETTINGS indexgranularity = 8192
```

In both cases, their Body fields were filled with data that simulates a JSON map object:

```
plaintext
CREATE FUNCTION genmap AS (n) -> arrayMap (x-> (x::String, (x(rand())%40000+1)::String),
range(1, n));

INSERT INTO tbl(2|3)
SELECT
  now() - randUniform(1, 1000000) as Timestamp,
  randomPrintableASCII(2) as Traceld,
  randomPrintableASCII(2) as ServiceName,
  rand32() as Duration,
  randomPrintableASCII(2) as SpanName,
  toJSONString(genmap(rand()%40+1)::Map(String, String)) as Body
FROM numbers(10000000);
```

In the case of the tokenbfv1 table, we have:

- == equality works, skipping index resulted in 224/1264 granules scanned:

```
plaintext
zara.engel.vespian.net :) explain indexes=1 select count() from tbl2 where Body == '{"1":"14732", "2":"29464", "3":"44196", "4":"58928", "5":"73660", "6":"88392", "7":"103124", "8":"117856", "9":"132588", "10":"147320", "11":"162052"}'
```

```
EXPLAIN indexes = 1
SELECT count()
FROM tbl2
WHERE Body = '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'
```

Query id: 60827945-a9b0-42f9-86a8-dfe77758a6b1

```
..explain.....
· Expression ((Projection + Before ORDER BY))
· Aggregating
· Expression (Before GROUP BY)
· Filter (WHERE)
· ReadFromMergeTree (logging.tbl2)
· Indexes:
· MinMax
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· Partition
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· PrimaryKey
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· Skip
· Name: idxbody
· Description: tokenbfv1 GRANULARITY 1
· Parts: 62/69
· Granules: 224/1264
.....
```

23 rows in set. Elapsed: 0.019 sec.

- != inequality works as well, but results in fulltext scan - all granules were scanned:

```
plaintext
zara.engel.vespian.net :) explain indexes=1 select count() from tbl2 where Body != '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'
```

```
EXPLAIN indexes = 1
SELECT count()
FROM tbl2
WHERE Body != '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'
```

Query id: 01584696-30d8-4711-8469-44d4f2629c98

```
..explain.....
· Expression ((Projection + Before ORDER BY))
· Aggregating
· Expression (Before GROUP BY)
· Filter (WHERE)
· ReadFromMergeTree (logging.tbl2)
· Indexes:
· MinMax
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· Partition
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· PrimaryKey
· Condition: true
· Parts: 69/69
· Granules: 1264/1264
· Skip
· Name: idxbody
· Description: tokenbfv1 GRANULARITY 1
· Parts: 69/69
· Granules: 1264/1264
.....
```

23 rows in set. Elapsed: 0.017 sec.

- LIKE queries work, 271/1264 granules scanned:

plaintext

zara.engel.vespian.net :) explain indexes=1 select from tbl2 where Body like
'%"11":"162052"%';

```
EXPLAIN indexes = 1
SELECT
FROM tbl2
WHERE Body LIKE '%"11":"162052"%'
```

Query id: 86e99d7a-6567-4000-badc-d0b8b2dc8936

```
..explain.....
· Expression ((Projection + Before ORDER BY))
· ReadFromMergeTree (logging.tbl2)
· Indexes:
· MinMax
· Condition: true
```

- Parts: 69/69
- Granules: 1264/1264
- Partition
- Condition: true
- Parts: 69/69
- Granules: 1264/1264
- PrimaryKey
- Condition: true
- Parts: 69/69
- Granules: 1264/1264
- Skip
- Name: idxbody
- Description: tokenbfv1 GRANULARITY 1
- Parts: 64/69
- Granules: 271/1264
-

20 rows in set. Elapsed: 0.047 sec.

ngrambfv1 tokenizer will be much harder to tune and use correctly:

- equality using n-gram indexes works as well, but due to the high granularity of tokens in the bloom filter, we aren't skipping many granules:

plaintext

zara.engel.vespian.net :) explain indexes=1 select count() from tbl3 where Body == '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'

```
EXPLAIN indexes = 1
SELECT count()
FROM tbl3
WHERE Body = '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'
```

Query id: 22836e2d-5e49-4f51-b23c-facf5a3102c2

```
..explain.....
· Expression ((Projection + Before ORDER BY))
· Aggregating
· Expression (Before GROUP BY)
· Filter (WHERE)
· ReadFromMergeTree (logging.tbl3)
· Indexes:
· MinMax
· Condition: true
· Parts: 60/60
· Granules: 1257/1257
· Partition
```

```

.      Condition: true      .
.      Parts: 60/60        .
.      Granules: 1257/1257 .
.      PrimaryKey          .
.      Condition: true     .
.      Parts: 60/60        .
.      Granules: 1257/1257 .
.      Skip                .
.      Name: idxbody       .
.      Description: ngrambfv1 GRANULARITY 1 .
.      Parts: 60/60        .
.      Granules: 1239/1257 .
.....

```

23 rows in set. Elapsed: 0.025 sec.

- inequality here also results in a full scan:

plaintext

```

zara.engel.vespian.net :) explain indexes=1 select count() from tbl3 where Body != '{"1":"14732",
"2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"13
2588","10":"147320","11":"162052"}'

```

EXPLAIN indexes = 1

SELECT count()

FROM tbl3

WHERE Body != '{"1":"14732","2":"29464","3":"44196","4":"58928","5":"73660","6":"88392","7":"103124","8":"117856","9":"132588","10":"147320","11":"162052"}'

Query id: 2378c885-65b0-4be0-9564-fa7ba7c79172

..explain.....

```

. Expression ((Projection + Before ORDER BY)) .
. Aggregating .
. Expression (Before GROUP BY) .
. Filter (WHERE) .
. ReadFromMergeTree (logging.tbl3) .
. Indexes: .
.   MinMax .
.   Condition: true .
.   Parts: 60/60 .
.   Granules: 1257/1257 .
. Partition .
.   Condition: true .
.   Parts: 60/60 .
.   Granules: 1257/1257 .
. PrimaryKey .
.   Condition: true .
.   Parts: 60/60 .

```

- Granules: 1257/1257
- Skip
- Name: idxbody
- Description: ngrambfv1 GRANULARITY 1
- Parts: 60/60
- Granules: 1257/1257

.....

23 rows in set. Elapsed: 0.022 sec.

- LIKE statements work, but result in a fullscan as the ngrams match all the granules:

```
plaintext
zara.engel.vespian.net :) explain indexes=1 select  from tbl3 where Body like
'%"11":"162052"%';
```

```
EXPLAIN indexes = 1
SELECT
FROM tbl3
WHERE Body LIKE '%"11":"162052"%'
```

Query id: 957d8c98-819e-4487-93ac-868ffe0485ec

```
..explain.....
. Expression ((Projection + Before ORDER BY)) .
. ReadFromMergeTree (logging.tbl3) .
. Indexes: .
.   MinMax .
.   Condition: true .
.   Parts: 60/60 .
.   Granules: 1257/1257 .
. Partition .
.   Condition: true .
.   Parts: 60/60 .
.   Granules: 1257/1257 .
. PrimaryKey .
.   Condition: true .
.   Parts: 60/60 .
.   Granules: 1257/1257 .
. Skip .
.   Name: idxbody .
.   Description: ngrambfv1 GRANULARITY 1 .
.   Parts: 60/60 .
.   Granules: 1251/1257 .
```

.....

20 rows in set. Elapsed: 0.023 sec.

Data Deduplication

To provide cost-efficient service to our users, we need to think about deduplicating the data we get from our users.

ClickHouse ReplacingMergeTree deduplicates data automatically based on the primary key. We can't include all the relevant Log entry fields in the primary field, hence the idea of a Fingerprint as the very last part of the Primary Key.

We normally do not use it for indexing, just to prevent unique records from being garbage collected.

The fingerprint calculation algorithm and length have not been chosen yet, we may use the same one that metrics are using to calculate their Fingerprint.

For now, we assume that it is 128-bit wide (16 8-bit chars).

The columns we use for fingerprint calculation are the columns that are not present in the primary key: Body, ResourceAttributes, and LogAttributes.

The fingerprint, due to very high cardinality, will need to go into the last place in the primary index.

Data Retention

There is a legal question of how long logs need to be stored and whether we allow for their deletion (e.g. due to the leak of some private data or data related to an investigation).

In some jurisdictions, logs need to be kept for years and there must be no way to delete them.

This affects deduplication unless we include the ObservedTimestamp in the fingerprint.

As pointed out in the Non-Goals section, this is an issue we are going to tackle in future iterations.

Ingestion-time fields

I am intentionally not pulling semantic convention fields into separate columns as users will use countless number of log formats, and it will probably not be possible to identify properties worth becoming a column.

The ObservedTimestamp field is set by the collector during the ingestion.

Users query by the Timestamp field and the log pruning is driven by the ObservedTimestamp field.

The disadvantage of this approach is that TTL DELETE may not remove parts as early as we would like to because the primary index and TTL column differ so the data may not be localized.

This seems like a good tradeoff though.

We will offer users a predefined storage period that starts with the ingestion.

In case when users ingest logs that have timestamps in the future or the past, the pruning of old logs could start too early or too late.

Users could abuse the claimed log timestamp too to delay pruning.

The ObservedTimestamp approach does not have these issues.

During the ingestion, the SeverityText field is parsed into SeverityNumber if the SeverityNumber field has not been set.

Queries will be using the SeverityNumber field as it is more efficient than plain text and offers higher granularity.

plaintext

```

DROP TABLE if exists logs;
CREATE TABLE logs
(
    ProjectId String CODEC(ZSTD(1)),
    Fingerprint FixedString(16) CODEC(ZSTD(1)),
    Timestamp DateTime64(9) CODEC(Delta(8), ZSTD(1)),
    ObservedTimestamp DateTime64(9) CODEC(Delta(8), ZSTD(1)),
    TraceId FixedString(16) CODEC(ZSTD(1)),
    SpanId FixedString(8) CODEC(ZSTD(1)),
    TraceFlags UInt32 CODEC(ZSTD(1)),
    SeverityText LowCardinality(String) CODEC(ZSTD(1)),
    SeverityNumber UInt8 CODEC(ZSTD(1)),
    ServiceName String CODEC(ZSTD(1)),
    Body String CODEC(ZSTD(1)),
    ResourceAttributes Map(LowCardinality(String), String) CODEC(ZSTD(1)),
    LogAttributes Map(LowCardinality(String), String) CODEC(ZSTD(1)),
    INDEX idxtraceid TraceId TYPE bloomfilter(0.001) GRANULARITY 1,
    INDEX idxspanid SpanId TYPE bloomfilter(0.001) GRANULARITY 1,
    INDEX idxtraceflags TraceFlags TYPE set(2) GRANULARITY 1,
    INDEX idxresattrkey mapKeys(ResourceAttributes) TYPE bloomfilter(0.01) GRANULARITY 1,
    INDEX idxresattrvalue mapValues(ResourceAttributes) TYPE bloomfilter(0.01) GRANULARITY 1,
    INDEX idxlogattrkey mapKeys(LogAttributes) TYPE bloomfilter(0.01) GRANULARITY 1,
    INDEX idxlogattrvalue mapValues(LogAttributes) TYPE bloomfilter(0.01) GRANULARITY 1,
    INDEX idxbody Body TYPE tokenbfv1(143776, 10, 0) GRANULARITY 1
)
ENGINE = ReplacingMergeTree
PARTITION BY toDate(Timestamp)
ORDER BY (ProjectId, ServiceName, SeverityNumber, toUnixTimestamp(Timestamp), TraceId, Fingerprint)
TTL toDate(Timestamp) + toIntervalDay(30)
SETTINGS indexgranularity = 8192, ttlonlydropparts = 1;

```

Query API, querying UI

The main idea behind query API/workflow introduced by this proposal is to give users the freedom to query while at the same time providing limits both when it comes to query complexity and query resource usage/execution time.

We can't foresee how users are going to query their data, nor how the data will look exactly - some will use Attributes, some will just focus on log-level, etc...

In Clickhouse, individual queries may have settings, which include query complexity settings. The query limits would be appended to each query automatically by the query service when constructing SQL statements.

Fulltext queries for the Log entry Body field would be handled transparently by the query service as well thanks to ClickHouse optimizing LIKE queries using BloomFilters and tokenization of the search term.

In future iterations we may want to consider n-gram tokenization, for now, the queries will be limited to full words only.

It is up for debate whether we want to deduplicate log entries in the UI in case the user ingests duplicates.

We could use the `max(ObservedTimestamp)` function to avoid duplicated entries in the time between records ingestion and `ReplacingMergeTree`'s eventual deduplication kicking in. Definitely not in the first iteration though.

The query service would also transparently translate the `SeverityText` attributes of the query into `SeverityNumber` while constructing the query.

Query Service API schema

We can't allow UI to send us SQL queries as that would open the possibility of abusing the system by users.

We are also unable to support all the use cases that users could come up with when given the full flexibility of SQL query language.

So the idea is for UI to provide a simple creator-like experience that would guide users.

Something very similar to what GitLab currently has for searching MRs and Issues.

UI code would then translate the query that the user came up with into a JSON and send it for processing to the query service.

Based on the JSON received, the query service would then template an SQL query together with the query limits we mentioned above.

For now, the UI and the JSON API would support only a basic set of operations on given fields:

- Timestamp: `>`, `<`, `==`
- TraceId: `==`, later iterations in
- SpanId: `==`, later iterations in
- TraceFlags: `==`, `!=`, later iterations: `in`, `notIn`
- SeverityText: `==`, `!=`, later iterations: `in`, `notIn`
- SeverityNumber: `<`, `>`, `==`, `!=`, later iterations: `in`, `notIn`
- ServiceName: `==`, `!=`, later iterations: `in`, `notIn`
- Body: `==`, `CONTAINS`
- ResourceAttributes: `key==value`, `mapContains(key)`
- LogAttributes: `key==value`, `mapContains(key)`

The format of the intermediate JSON could look like the following:

```
yaml
{
  "query": [
    { "type": "()"|AND|OR",
      "operands": {
        [...]
      },
    {
      "type": "==|!=|<|>|CONTAINS",
      "column": "...",
      "val": "..."
    }
  ]
}
```

}

The ==|!=|<|>|CONTAINS are non-nesting operands, they operate on concrete columns and result in WHEN conditions after being processed by the query service.

The ()|AND|OR are nesting operands and can only include other non-nesting operands.

We may defer the implementation of the nesting operands for later iterations.

There is implicit AND between the operands at the top level of the query structure.

The query schema is intentionally kept simple compared to the one used in the metrics proposal. We may add fields like QueryContext, BackendContext, etc... in later iterations once a need arises.

For now, we keep the schema as simple as possible and just make sure that the API is versioned so that we can change it easily in the future.

Open questions

Logging SDK Maturity

OTEL standard does not intend to provide a standalone SDK for logging just like it did e.g. tracing.

It may consider doing so only for a programming language that does not have its logging libraries which should be a pretty rare thing.

All the existing logging libraries should instead use bridge API to interact with OTEL collector/send logs using OTEL Logs standard.

The majority of languages have already made the required adjustments, except for Go.

There is only very minimal support for GO (repo, repo).

The official Uber Zap repository has barely an issue about emitting events in spans.

Opentelemetry status page states that Go support as not implemented yet.

The lack of native OTEL SDK support for emitting logs in Go may be an issue for us if we want to dogfood logging.

We could work around these limitations in large extent by parsing log files using filelogreceiver or fluentd.

Contributing and improving the support of Go in OTEL is also a valid option.

Future work

Support for != operators in queries

Bloom filters that we use in schemas do not allow for testing if the given term is NOT present in the log entry's body/attributes.

This is a small but valid use case.

A solution for that may be inverted indexes but this is still an experimental feature.

Documentation

As part of the documentation effort, we may want to provide examples of how sending data to GOB can be done in different languages (uber-zap, logrus, log4j, etc...) just like we do for error

tracking.

Some of the applications can't be easily modified to send data to us (e.g. systemd/journald) and a log tailing/parsing needs to be employed using filelogreceiver or fluentd.

We could probably address both cases above by instrumenting our infrastructure and linking to our code from the documentation.

This way we can both dog-food our solution, save some money as the GCE logging solution is pretty expensive, and give users real-life examples of how they can instrument their infrastructure.

This could be one of the follow-up tasks once we are done with the implementation.

User query resource usage monitoring

Long-term, we will need a way to monitor the number of user queries that failed due to limits enforcement and resource usage in general to fine-tune the query limits and make sure that users are not too aggressively restricted.

Iterations

Refer to Observability Group planning epic and its linked issues for up-to-date information.

SECTION: engineering/architecture/design-documents/observability_metrics/_index.md

{{< engineering/design-document-header >}}

Summary

Developing a multi-user system to store & query observability data typically formatted in widely accepted, industry-standard formats such as OpenTelemetry using Clickhouse as the underlying storage with support for long-term data retention and aggregation.

Motivation

From the six pillars of Observability, commonly abbreviated as TEMPLE - Traces, Events, Metrics, Profiles, Logs & Errors, Metrics constitute one of the most important of those for modern day systems helping their users gather insights about the operational posture of monitored systems.

Metrics which are commonly structured as timeseries data have the following characteristics:

- indexed by their corresponding timestamps;
- continuously expanding in size;
- usually aggregated, down-sampled, and queried in ranges; and
- have very write-intensive requirements.

Within GitLab Observability Backend, we aim to add the support for our customers to ingest and query observability data around their systems & applications, helping them improve the operational health of their systems.

Goals

With the development of the proposed system, we have the following goals:

- Scalable, low latency & cost-effective monitoring system backed by Clickhouse whose performance has been proven via repeatable benchmarks.
- Support for long-term storage for metrics, ingested via an OpenTelemetry-compliant agent and queried via GitLab-native UI with probable support for metadata and exemplars.

The aforementioned goals can further be broken down into the following four sub-goals:

Ingesting data

- For the system to be capable of ingesting large volumes of writes and reads, we aim to ensure that it must be horizontally scalable & provide durability guarantees to ensure no writes are dropped once ingested.

Persisting data

- We aim to support ingesting telemetry/data instrumented using OpenTelemetry specifications. For a first iteration, any persistence we design for our dataset will be multi-tenant by default, ensuring we can store observability data for multiple groups/projects within the same storage backend.

Reading data

- We aim to support querying data via a GitLab-native UX which would mean using a custom DSL/Query Builder sending API requests to our backend which would then translate them into Clickhouse SQL. From our internal discussions around this, Product Analytics Visualisation Designer is a good source of inspiration for this.

Deleting data

- We aim to support being able to delete any ingested data should such a need arise. This is also in addition to us naturally deleting data when a configured TTL expires and/or respective retention policies are enforced. We must, within our schemas, build a way to delete data by labels OR their content, also add to our offering the necessary tooling to do so.

Non-Goals

With the goals established above, we also want to establish what specific things are non-goals with the current proposal. They are:

- With our first iteration here, we do not aim to support querying ingested telemetry via PromQL deferring that to as & when such a business need arises. However, users will be able to ingest their metrics using the OpenTelemetry Line Protocol (OTLP), e.g. via the Prometheus Receiver in case of Prometheus metrics.

Proposal

We intend to use GitLab Observability Backend (GOB) as a framework for the Metrics implementation so that its lifecycle can be managed via already established components of our backend.

As depicted in the diagram above, an OTEL-collector pipeline, indexer & query service are components that need to be developed as proposed here while the remaining peripheral components either already exist or can be provisioned via existing code in our centralised scheduler within GOB.

On the write path:

- We expect to receive incoming data via HTTP/JSON similar to what we do for our existing services, e.g. errortracking, tracing.
- We aim to heavily deduplicate incoming timeseries by indexing/caching per-series metadata to reduce our storage footprint.
- We aim to ensure avoiding writing a lot of small writes into Clickhouse by batching data before writing it into Clickhouse.

On the read path:

- We aim to allow our users to use GitLab itself to read ingested data, which will necessitate building a dedicated Query Service on our backend to be able to service API requests originating from GitLab.
- We aim implement necessary query validation, sanitation and rate-limiting for any resource consumption to ensure underlying systems remain in good operational health at all times.

GitLab Observability Tenant

With the recent changes to our backend design especially around deprecating the use of a Grafana-based UX, we have found opportunities to streamline how we provision tenants within our system. This initiative had led to the development of a custom CR - `GitLabObservabilityTenant` intended to model a dedicated set of resources per top-level GitLab namespace. From a scalability perspective, this means we deploy a dedicated instance of Ingress & Ingestor per top-level GitLab namespace to make sure we can scale each tenant subject to traffic volumes of its respective groups & projects. It also helps isolate resource consumption across tenants in an otherwise multi-tenant system such as ours.

Indexing per-series metadata

As an internal part of the ingestor, we aim to index per-series labels and/or metadata to be able to deduplicate incoming timeseries data and segregate them into metadata and points-data. This helps reduce our storage footprint by an order of magnitude keeping total cost of operation low. This indexed data can also be consumed by the Query Service to efficiently

compute timeseries for all incoming read requests. This part of our architecture is also described in more detail in Proposal: Indexing metrics labels for efficiently deduplicating & querying time series data.

Query Service

The Query Service consists of two primary components - 1. a request parser & 2. a backend-specific querier implementation. On the request path, once its received on the designated endpoint(s), it is handled by a handler which is a part of the request parser. The parser's responsibility is to unmarshal incoming query payloads, validate the contents and produce a SearchContext object which describes how must this query/request be processed. Within a SearchContext object is a QueryContext attribute which further defines one or more Query objects - each a completely independent data query against one of our backends.

API structure

For the user-facing API, we intend to add support via HTTP/JSON endpoint(s) with user-queries marshalled as payloads within a request body. For example, to compute the sum of a minutely delta of metric:apiserverrequesttotal over all values of label:instance, you'd send a POST request to [https://observe.gitlab.com/query/\\$GROUP/\\$PROJECT/metrics](https://observe.gitlab.com/query/$GROUP/$PROJECT/metrics) with the following as body:

```
json
{
  "queries": {
    "A": {
      "type": "metrics",
      "filters": [
        {
          "key": "name",
          "value": "apiserverrequesttotal",
          "operator": "eq"
        }
      ],
      "aggregation": {
        "function": "rate",
        "interval": "1m"
      },
      "groupBy": {
        "attribute": [
          "instance"
        ],
        "function": "sum"
      },
      "sortBy": {},
      "legend": {}
    }
  },
  "expression": "A"
```



```
}
```

Query representation as an AST

plaintext

```
type SearchContext struct {
    UserContext    UserContext    json:"authContext"
    BackendContext BackendContext json:"backendContext"

    StartTimestamp int64 json:"start"
    EndTimestamp   int64 json:"end"
    StepIntervalSeconds int64 json:"step"

    QueryContext    QueryContext    json:"queryContext"
    CorrelationContext CorrelationContext json:"correlationContext"
    Variables       map[string]interface{} json:"variables,omitempty"
}
```

Generally speaking:

- SearchContext defines how a search must be executed.
 - It internally contains a QueryContext which points to one or more Query(s) each targeting a given backend.
 - Each Query must be parsed & processed independently, supplemented by other common attributes within a QueryContext or SearchContext.
- Query defines an AST-like object which describes how must a query be performed.
 - It is intentionally schema-agnostic allowing it to be serialised and passed around our system(s).
 - It is also an abstraction that hides details of how we model data internal to our databases from the querying entity.
 - Assuming an incoming query can be parsed & validated into a Query object, a Querier can execute a search/query against it.
- UserContext defines if a request has access to the data being searched for.
 - It is perhaps a good place to model & enforce request quotas, rate-limiting, etc.
 - Populating parts of this attribute depend on the parser reading other global state via the API gateway or Gatekeeper.
- BackendContext defines which backend must a request be processed against.
 - It helps route requests to an appropriate backend in a multitenant environment.
 - For this iteration though, we intend to work with only one backend as is the case with our architecture.
- CorrelationContext defines how multiple queries can be correlated to each other to build a cohesive view on the frontend.
 - For this iteration though, we intend to keep it empty and only work on adding correlation vectors later.

Intended target-environments

Keeping inline with our current operational structure, we intend to deploy the metrics offering as a part of GitLab Observability Backend, deployed on the following two target environments:

- kind cluster (for local development)
- GKE cluster (for staging/production environments)

Production Readiness

Batching

Considering we'll need to batch data before ingesting large volumes of small writes into Clickhouse, the design must account for app-local persistence to allow it to locally batch incoming data before landing it into Clickhouse in batches of a predetermined size in order to increase performance and allow the table engine to continue to persist data successfully.

We have considered the following alternatives to implement app-local batching:

- In-memory - non durable
- BadgerDB - durable, embedded, performant
- Redis - trivial, external dependency
- Kafka - non-trivial, external dependency but it can augment multiple other use-cases and help other problem domains at GitLab.

Note: Similar challenges have also surfaced with the CH interactions errortracking - the subsystem has in its current implementation. There have been multiple attempts to solve this problem domain in the past - this MR implemented an in-memory alternative while this one attempted an on-disk alternative.

Any work done in this area of concern would also benefit other subsystems such as errortracking, logging, etc.

Scalability

We intend to start testing the proposed implementation with 10K metric-points per second to test/establish our initial hypothesis, though ideally, we must design the underlying backend for 1M points ingested per second.

Benchmarking

We propose the following three dimensions be tested while benchmarking the proposed implementation:

- Data ingest performance (functional)
- Mean query response times (functional)
- Storage requirements (operational)

For understanding performance, we'll need to first compile a list of such queries given the data we ingest for our tests. Clickhouse query logging is super helpful while doing this.

NOTE:

Ideally, we aim to benchmark the system to be able to ingest >1M metric points/sec while consistently serving most queries under <1 sec.

Past work & references

- Benchmark ClickHouse for metrics
- Incubation:APM ClickHouse evaluation
- Incubation:APM ClickHouse metrics schema
- Our research around TimescaleDB
- Current Workload on our Thanos-based setup
- Scaling-200m-series

Cost-estimation

- We aim to make sure the system is cost-effective to our users for ingesting & querying telemetry data. One of the more significant factors affecting underlying costs are how we model & store ingested data which the intended proposal must optimize for by measures such as reducing data redundancy, pruning unused metrics, etc.
- We must consider the usage of multiple storage medium(s), especially:
 - Tiered storage
 - Object storage

Tooling

As an overarching outcome here, we aim to build the necessary tooling and/or telemetry around ingested data to enable all user personas to have visibility into high cardinality metrics to help prune or drop unused metrics. It'd be prudent to have usage statistics e.g. per-metric scrape frequencies, to make sure our end-users are not ingesting data at a volume they do not need and/or find useful.

Future iterations

Linkage across telemetry pillars, exemplars

We must build the metrics system in a way to be able cross-reference ingested data with other telemetry pillars, such as traces, logs and errors, so as to provide a more holistic view of all instrumentation a system sends our way.

Support for user-defined SQL queries to aggregate data and/or generate materialized views

We should allow users of the system to be able to run user-defined, ad-hoc queries similar to how Prometheus recording rules help generate custom metrics from existing ones.

Support for scalable data ingestion

We believe that should we feel the need to start buffering data local to the ingestion application and/or move away from Clickhouse for persisting data, on-disk WALs would be a good direction to proceed into given their prevalent usage among other monitoring systems.

Query Service features

- Adding support for compound queries and/or expressions.
- Consolidation of querying capabilities for tracing, logs & errortracking via the query engine.
- Using the query engine to build integrations such as alerting.
- Adding support for other monitoring/querying standards such as PromQL, MetricQL, OpenSearch, etc
- Adding automated insights around metric cardinality & resource consumption.

Planned roadmap

The following section enlists how we intend to implement the aforementioned proposal around building Metrics support into GitLab Observability Service. Each corresponding document and/or issue contains further details of how each next step is planned to be executed.

16.5

- Research & draft design proposal and/or requirements.
- Produce architectural blueprint, open for feedback.

16.6

- Develop support for OpenTelemetry-based ingestion.
- Develop support for querying data; begin with an API to list all ingested metrics scoped to a given tenant.
- Develop support for displaying a list of ingested metrics within GitLab UI.
- Release experimental version.

16.7

- Develop support for querying data, add metrics search endpoints for supported metric-types.
- Develop our first iteration of the query builder, enable querying backend APIs.
- Develop a metrics details page with the ability to graph data returned via backend APIs.
- Setup testing, ensure repeatable benchmarking/testing can be performed.
- Release Beta version, open for early usage by internal and external customers.

16.9 (Gap to allow for user feedback for GA release)

- Develop end-to-end testing, complete necessary production readiness, address feedback from users.
- Release GA version.

SECTION: engineering/architecture/design-documents/observability_tracing/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab already has distributed tracing as a feature. So this proposal focuses on the intended changes required to GA the feature. Given the strategic direction update which is covered more in the motivation section, we are deprecating the GitLab Observability UI (GOUI) in favor of building native UI for tracing in GitLab UI.

This proposal covers the scope and technical approach to what will be released in GA, including the new UI, API changes and any backend changes to support the new direction.

Distributed Tracing will GA as a premium feature, initially only available to premium and ultimate users.

Motivation

In December 2021 GitLab acquired OpsTrace and kicked off work integrating Observability functionality into the DevOps platform. At that point the stated goal was to create an observability distribution that could be run independently of GitLab and which integrated well into the DevSecOps platform. See Internal Only- Argus FAQ for more background on previous strategy.

Since December 2021 there have been a lot of changes in the world and at GitLab. It is GitLab's belief that Observability should be natively built within GitLab UI to avoid fracturing capabilities and ensuring a singular UX. As such we are deprecating GitLab Observability UI which began life as a fork of Grafana in December 2021.

Much of the GitLab Observability architecture and features were built around the fork of Grafana. As such, this proposal is part of a series of proposals that align us toward achieving the following high level objectives.

Observability Group Objectives

The following group-level objectives are included for context. The Objectives below are not completely covered by this design. This design focuses on distributed tracing. More design documents will be created for logging, metrics, auto-monitor, etc.

Timeline: Completion of the following before December 2024

Objectives:

- GA of a Complete (Metrics, Logs, Traces) Observability Platform - Add on-by-default setup for tracing, metrics and logging including a GA for the service available on GitLab.com and for self-managed users. A user is able to trace micro-services or distributed systems using open-source tracers. Furthermore, users should be able to set sane defaults for sampling or use advanced techniques such as tail-based sampling.
- Tailored Triage Workflow - Users need to connect the dots between Metrics, Logs, and Spans/Traces. Designing for the discovery, querying and the connection of all telemetry data, regardless of type, will aid users to resolve critical alerts and incidents more quickly.
- Auto Monitor - When a developer starts a new project their application is automatically instrumented, alerts are set up and linked to GitLab alerts management, schedules are created

and incidents are created for critical alerts.

Goals

To release a generally available distributed tracing feature as part of GitLab.com SaaS with a minimum featureset such that it is valuable but can be iterated upon.

Specific goals:

- An HTTPS write API implemented in the GitLab Observability Backend project which receives spans sent to GitLab using OTLP (OpenTelemetry Protocol). Users can collect and send distributed traces using either the OpenTelemetry SDK or the OpenTelemetry Collector.
- UI to list and filter/search for traces by ID, service, attributes or time
- UI to show a detail view of a trace and its corresponding spans
- Apply sensible ingestion and storage limits per top-level namespace for all GitLab tiers

Timeline

In order to achieve the group objectives, the following timelines must be met for GitLab phased rollout of Tracing.

- Tracing Experiment Release: 16.2
- Tracing Beta Release: 16.3
- Tracing GA Release: 16.4

Proposal

Much of the proposed architecture already exists and is in operation for GitLab.com. Distributed tracing has already been in an internal Beta for quite some time and has internal users, with graduation to GA being blocked by UX requirements. These UX requirements resulted in the new UI strategy.

The following diagram outlines the architecture for GitLab Observability Backend and how clients, including the GitLab UI, will interact with it.

>

Key Components

- Gatekeeper: Responsible for authentication, authorization and rate limit enforcement on all incoming requests. NGINX-Ingress interacts directly with Gatekeeper.
- Ingress: NGINX-Ingress is used to handle all incoming requests
- ClickHouse: ClickHouse is the backing store for all observability data
- Query Service: A horizontally scalable service that retrieves data from ClickHouse in response to a query
- GitLab UI: The UI hosted at GitLab.com
- Redis: An HA Redis cluster for caching GitLab API responses

Data Ingest

One data ingestion pipeline will be deployed for each top level GitLab namespace. Currently we deploy one pipeline per GitLab Group that enables observability and this architecture is now unnecessarily expensive and complex without the presence of the multi-tenant Grafana instances. This multi-tenant ingestion system has the following benefits:

- Beyond rate limits, resource limits can be enforced per user such that no user can steal more system resources (memory, cpu) than allocated.
- Fine grained control of horizontal scaling for each user pipeline by adding more OTEL Collector instances
- Manage the users tenant in accordance to GitLab subscription tier, for example, quota, throughput, cold storage, shard to different databases
- Reduced complexity and enhanced security in the pipeline by leveraging off the shelf components like the OpenTelemetry Collector where data within that collector belongs to no more than a single user/customer.

A pipeline is only deployed for the user upon enabling observability in the project settings, in the same way a user can enable error tracking for their project. When observability is enabled for any project in the users namespace, a pipeline will be deployed. This deployment is automated by our Kubernetes scheduler-operator and tenant-operator. Provisioning is currently managed through the iframe, but a preferred method would be to provision using a RESTful API. The GitLab UI would have a section in project settings that allow a user to "enable observability", much like they do for error tracking today.

The opentelemetry collector is used as the core pipeline implementation for its excellent community development of receivers, processors and exporters. An exporter for ClickHouse has emerged in the community which we intend to leverage and it currently has support for opentelemetry traces, metrics and logs. This will help accelerate the effort toward ingesting not just traces but also metrics and logs.

Limits

In addition to the existing cpu and memory limits for each ingest pipeline, the following limits and quotas will also be enforced:

- 100KB (possibly increase this to 1MB) total ingest rate of traces per second per top level namespace
- 30 day data retention
- TBD GB total storage

All above limits are subject to change and will be driven by top level namespace configuration so scripts and future features can be built to make these more dynamic for each user or subscription tier. This configuration will be part of the tenant-operator custom resource.

The ingest rate limit will utilize the internal Redis cluster to perform a simple, performant sliding window rate limit like Cloudflare. The code for this will live in Gatekeeper, where a connection to Redis is already managed.

The data retention and total storage limits will be enforced by a control loop in the tenant-operator that will periodically query ClickHouse and continuously delete the oldest whole day of data until quota is no longer exceeded. To do this efficiently, its important that

ClickHouse tables are partitioned using `toDate(timestamp)` to partition by day.

Query API

The query API, backed by the query service, will be a centralized, horizontally scalable component responsible for returning traces/spans back to the UI. A good starting point for this query service may be to leverage the Jaeger query service code and the Jaeger query service swagger. This query service will be extended to include support for metrics and logs in the future and will be queried directly by vue.js code in GitLab UI.

The scope of effort for GA would include two APIs:

- `/v1/traces` adhering to this spec
- `/v1/traces/{traceID}` adhering to this spec

Authentication and Authorization

<!-- markdownlint-disable-next-line MD044 -->

GitLab Observability Backend utilizes an instance-wide trusted GitLab OAuth token to perform a seamless OAuth flow that authenticates the GitLab user against the GitLab Observability Backend (GOB). GOB creates an auth session and stores the session identifier in an http-only, secure cookie. This mechanism has already been examined and approved by AppSec. Now that the Observability UI will be native within the UI hosted at GitLab.com, a few small adjustments must be made for authentication to work against the new UI domain vs the embedded iframe that we previously relied upon (GitLab.com instead of `observe.gitLab.com`).

A hidden iframe will be embedded in the GitLab UI only on pages where GOB authenticated APIs must be consumed. This allows GitLab.com UI to directly communicate with GOB APIs without the need for an intermediate proxy layer in rails and without relying on the less secure shared token between proxy and GOB. This iframe will be hidden and its sole purpose is to perform the OAuth flow and assign the http-only secure cookie containing the GOB user session. This flow is seamless and can be fully hidden from the user since its a trusted GitLab OAuth flow. Sessions currently expire after 30 days which is configurable in GOB deployment terraform.

<!-- markdownlint-disable-next-line MD044 -->

In order to allow requests from GitLab.com directly to `observe.gitLab.com` using this method, all requests will have to include `{withCredentials: true}` in order to include cookies. For these "readonly" APIs that GitLab.com will query for tracing data, we must:

- Configure Ingress with `"NGINX.Ingress.Kubernetes.io/cors-allow-credentials": "true"` and `"NGINX.Ingress.Kubernetes.io/cors-allow-origin": "GitLab.com"`
- Ensure we have effective CSRF protection enabled in our Gatekeeper component (Gatekeeper is responsible request authorization)

<!-- markdownlint-disable-next-line MD044 -->

All requests from GitLab.com will then include the GOB session cookie for `observe.gitLab.com` to validate. Authorization is handled by the Gatekeeper component which checks group/project membership against GitLab and handles access appropriately. Anyone with inherited developer or above membership will have access to the tracing UI for that project.

Database Schema

The community developed OTEL exporter for ClickHouse has already implemented database schemas for storing traces and spans. This blog post from ClickHouse further delves into the details of the community developed exporter and we intend to use the suggested schema design as a starting point for us to test during experiment and beta phases. It's recommended to read the blog post to learn more about the schemas and corresponding SQL queries we intend to try.

UI Design

The new UI will be built using the Pajamas Design System in accordance with GitLab UX design standards. The UI will interact with the GOB query service directly from vue.js (see architecture diagram above) by sending a fetch to the subdomain `observe.gitLab.com/v1/query` with `{withCredentials: true}`. See the Authentication and Authorization section above for more details on how this is enabled.

TODO Figma UI designs and commentary

Iterations

16.2

- migrate all resources attached to the Group CR to the Tenant CR
- fork and build Clickhouse exporter
- add projectID to all traces/spans
- gatekeeper: check membership at project level instead of group level
- basic query service for listing traces (no filtering/searching)
- implement hidden iframe-based OAuth mechanism (reuse/adapt what already done for GOUI)
- UI for traces list

16.3

- filtering/searching query service (by traceID, service, status, duration min/max, start/end time, span attributes)
- add readobservability and writeobservability scopes to Project access token and support Project access token for writing to project level ingest API
- provision API
- remove existing iframe provisioning
- UI for trace detail
- UI for filtering/searching traces
- basic e2e test for provision, send data, query in UI
- metrics, dashboards, alerts

16.4

- UI settings page to "enable observability" (this would interact with provisioning API)
- production readiness review
- documentation complete
- alter GitLabNamespace CR to only represent a tenant (i.e. top level namespace)
- delete Group CR and corresponding controller

- e2e tests that haven't been added yet
- in cluster smoke test

SECTION: engineering/architecture/design-documents/organization/_index.md

{{< engineering/design-document-header >}}

This document is a work in progress and represents the current state of the Organization design.

Glossary

- User: A user account.
- Member: A User that belongs to an entity with a set of privileges represented by a role. A User can be a Member of one Organization, and many Groups and Projects within said Organization.
- Top-level Group: Top-level Group is the name given to the topmost Group of all other Groups. Groups and Projects are nested underneath the top-level Group.
- Organization: An Organization is the container for one or multiple top-level Groups. Organizations are isolated from each other.
- Organization Member: Organizations have many Users called Members. Only Organization Members have visibility of the Organization. Adding a User to a Group or Project within an Organization makes them an Organization Member.
- Default Organization: An Organization with ID = 1 seeded on every GitLab instance.

Summary

GitLab.com is a public shared installation of the GitLab software. This provides GitLab as a convenient SaaS but falls short of the full GitLab experience in important ways:

1. Parity: The features provided to a customer on GitLab.com and Self Managed are different. For example, on GitLab.com customers do not receive administrative privileges which accounts for a significant amount of functionality.
2. Isolation: On GitLab.com a customer can not exist independent of other customers like they do with a Self Managed installation.

Organizations will solve these problems by being a common container across all platforms. Through the creation of an Organization container we can enforce isolation boundaries, and provide a common entity for all top level features.

In effect, the Organization will wrap the Self Managed features into a container and bring this experience to all other GitLab platforms.

The isolation solution is also a pre-requisite for the Cells project which is described in relation to Organization in Organizations and Cells.

Frequently Asked Questions

If you have a specific question it may be answered within our FAQ or you may also try to query GitLab Duo Chat referencing the "Organization Blueprints".

Splitting the GitLab.com Platform

The GitLab.com platform will be split into two distinct experiences.

Customers join GitLab.com today as a top level group within the default organization. This experience will persist indefinitely in part to allow for a shared pool of users to contribute to open source projects.

GitLab.com will now expand its offering with a solution for private enterprise Organizations. These enterprise Organizations will operate in complete isolation from all other Organizations, including the default organization.

Eventually it will be possible for customers to migrate out of the default organization and into their own private Organization.

Fundamentals of Organizations

- Organization will wrap around nearly all GitLab features.
- It won't be possible to read or write data between Organizations. Read more about Organization Isolation.
- Many product features will remain unchanged, but most instance level features will move down and other features up to Organization level. Level changes are elaborated below.
- Users can only be a Member of a single Organization.
- They can be owners of the Organization or just standard members.
- In future we will review the ability for Users to be a Member of multiple Organizations.
- Organization owners will have admin style privileges within their Organization, such as the ability to delete user accounts. More details below.
- These changes will occur on all GitLab platforms including GitLab.com, Self Managed, and Dedicated.

Organization Isolation

All Organization data and functionality in GitLab will be isolated. Isolation means that data and features can never cross Organization boundaries. This is covered in further detail at Organization Isolation.

Impact of the Organization on Other Domains

Here is a growing list of pages that describe in more detail how Organization affects other parts of the system.

- Billing
- Cells
- Settings
- Users

Level Structure

Organization will form a new level that combines most Instance Level functionality and all of the Top Level Group functionality.

Instance Level will be reserved for infrastructure level settings.

Top Level Groups are a psuedo level that overload Group level with specific behavior. This has been the defacto method of GitLab.com finding parity with Self Managed, until Organizations,

Below is a depiction of the current and future hierarchy levels within GitLab.

Current Hierarchy	Future Hierarchy
-----	-----
Instance Level	Instance Level
	Organization Level
Top Level Group	
Group	Group
Project	Project

Only core features will be moved to Organization prior to Organization launch. After launch all remaining features will move to the Organization level.

Here is an entity diagram of these levels:

```
mermaid
graph TD
  o[Organization] -. has many .-> g[Group]
  ns[Namespace] --> g
  ns[Namespace] --> pns[ProjectNamespace]
  pns -. has one .-> p[Project]
  ns --> un[UserNamespace]
  g -. has many .-> p
  un -. has many .-> p
  ns[Namespace] -. has many .-> ns[Namespace]
```

Roles and Permissions

Organizations will have an Owner role. Compared to other Organization Members, they can perform the following actions:

Action	Owner	Member
-----	-----	-----
View Organization settings	·	·
Edit Organization settings	·	·
Delete Organization	·	·
Remove Users	·	·
View Organization front page	·	·
View Groups overview	·	· (1)
View Projects overview	·	· (1)
View Users overview	·	·

| View Organization activity page | · | · (1) |
| Transfer top-level Group into Organization if Owner of both | · | |

(1) Members can only see what they have access to.

Roles at the Group and Project level remain as they currently are.

Relationship between Organization Owner and Instance Admin

Users with the (Instance) Admin role can currently administer a self-managed GitLab instance. As functionality is moved to the Organization level, Organization Owners will be able to access more features that are currently only accessible to Admins. On our SaaS platform, this helps us in empowering enterprises to manage their own Organization more efficiently without depending on the Instance Admin, which is currently a GitLab team member. On SaaS, we expect the Instance Admin and the Organization Owner to be different users. Self-managed instances are generally scoped to a single organization, so in this case it is possible that both roles are fulfilled by the same person. There are situations that might require intervention by an Instance Admin, for instance when Users are abusing the system. When that is the case, actions taken by the Instance Admin overrule actions of the Organization Owner. For instance, the Instance Admin can ban or delete a User on behalf of the Organization Owner.

Routing

Today only Users, Projects, Namespaces and container images are considered routable entities which require global uniqueness on https://gitlab.com/<path>/-. Initially, Organization routes will be unscoped. Organizations will follow the path <https://gitlab.com/-/organizations/org-name/> as one of the design goals is that the addition of Organizations should not change existing Group and Project paths.

Organization Development

Below is a high level development roadmap for Organizations. The project is complicated and requires coordination across many engineering teams. In response to this, the roadmap has been broken into the following broad phases.

mermaid

gantt

dateFormat YYYY-MM-DD

axisFormat X

todayMarker off

%% Dates are for illustration purposes only.

Backend Essentials: active, be, 2024-01-01, 2025-10-31

Organization Sharding : active, sh, 2024-01-01, 2025-10-31

Organization Product Feature : active, ui, 2024-01-01, 2027-02-01

Now : milestone, now, 2024-03-18,

Organization Scoping: active, sc, after now, 2025-10-31
Organization Isolation : milestone, 2025-11-01,

Organization Level Features : co, after now, 2027-02-01
Organization Released : milestone, 2026-02-01,

Full Parity : milestone, 2027-02-01,

Milestones

Now (During FY26 Q1)
: Work has carried over from the Cells roadmap.

Organization Isolation (Target FY26 Q3)
: Complete database sharding to link tables back to an Organization where possible.
: Prevent reads/writes from extending beyond Organization boundaries.

Organization Released (Target FY26 Q4)
: Organization as a minimal feature is released in the application. The nature of this release is still under discussion.

Full Parity (Target FY27 Q4 or beyond)
: GitLab.com will provide the same feature set as Self Managed and Dedicated but on a shared platform.

Work Streams

Backend Essentials

This is foundational work to integrate the Organization at low levels of the code base.
Ensure all Organizations are equal by removing references to the Default Organization (ID = 1).
This is in part a carry over from the 2023 roadmap.

Organization Sharding

Tables, with a small number of exceptions, should be related to an Organization.
Organizational tables must have an organizationid, namespaceid, or a projectid column so all tables directly or indirectly belong to an Organization.
This work is currently located within this epic: <https://gitlab.com/groups/gitlab-org/-/epics/13678>.
All tables with an organizationid foreign key are defined with not null foreign key constraints.
All code paths are writing the correct organizationid value and are not relying on a default value.

Organization Product Feature

Build a user inter